

improving™

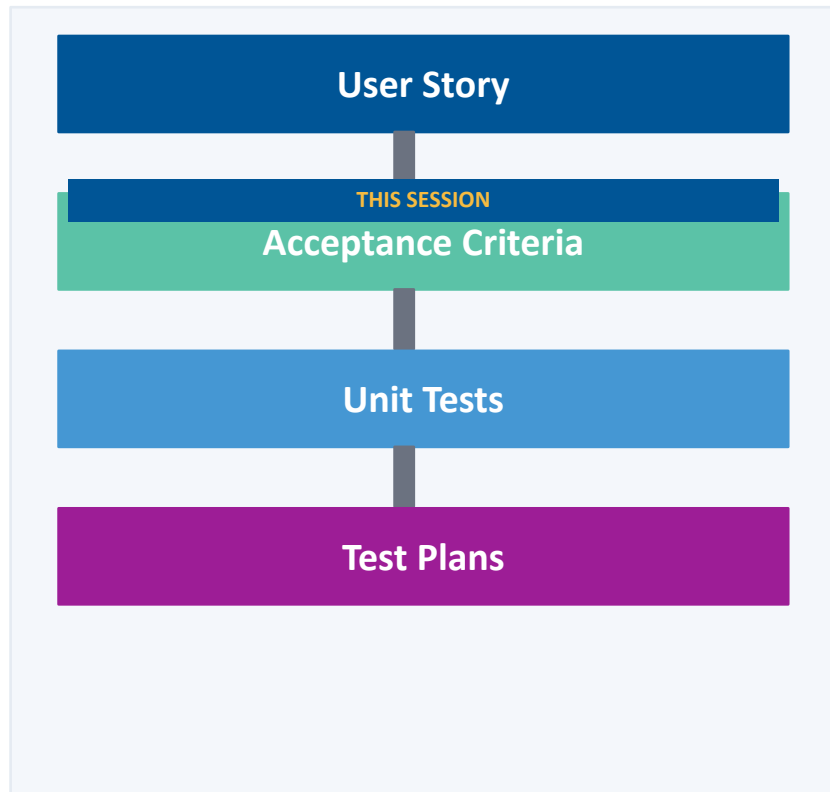
DEEP LEARNING PROGRAM

Training Session 1

Acceptance Criteria Generation

Week 1 | 4 Hours | All Participants | Loop 1 — Role Focus: Product/Process

© 2026 Improving | Confidential



Why This Session Comes First

The program starts here because Acceptance Criteria is the upstream artifact that determines quality everywhere downstream.

Vague AC → Vague Tests → Vague Test Plans. This chain is visible. Today we make it concrete.

- ✓ Understands what Basic Prompting vs. DLP Completion looks like in daily work.
- ✓ Used the RTCC structure (Role, Task, Context, Constraints) to build an Acceptance Criteria command
- ✓ Understands the Evaluation-Driven Development discipline: define success criteria before building any command
- ✓ Has generated Acceptance Criteria from at least 3 real stories and evaluated the output against written criteria
- ✓ Can apply the Load-Bearing Rule: audit each instruction in a command against its criteria
- ✓ Is ready to work with the Embedded Engineer tomorrow on real Claude skill and rule development and augmentation

Time	Block	Format
0:00–0:15	Basic Prompting vs. DLP Completion — the shortest version	Presentation
0:15–0:35	IDE setup + rules file	Hands-on Lab
0:35–0:50	What good Acceptance Criteria looks like — and what AI gets wrong	Demonstration
0:50–1:10	RTCC: the structure every command in this program uses	Presentation + Exercise
1:10–1:30	EDD: define success before you build	Presentation + Exercise
1:30–1:45	Break	
1:45–2:30	Build your Acceptance Criteria generation command	Hands-on Lab
2:30–3:00	Use it: Acceptance Criteria generation on real stories	Hands-on Lab
3:00–3:15	Break	
3:15–3:35	Evaluate the output and apply the Load-Bearing Rule	Discussion + Analysis
3:35–3:50	Baseline AI Adoption Maturity Assessment	Individual
3:50–4:00	Wrap-up: what to expect from the Embedded Engineer this week	Discussion

SESSION BLOCK

Basic Prompting vs. DLP Completion

Block 1 | 15 Minutes | Presentation

Basic Prompting

AI is a tool you pick up and put down
Used off-the-shelf, no structure
No repeatability or consistency
Output quality depends on the day

DLP Completion

AI is embedded in how you work
Skill files, evaluation, iteration
Transferable and repeatable craft
The difference is deliberate practice

Lab 1: IDE Setup + Rules File

20 MINUTES

Hands-on Lab

Create a rules file and confirm you are set up for the next 12 weeks.

1

Open your text editor and Coding Agent

You must be able to create markdown files and talk to your agent.

2

Create a project-level rules file

Put an AGENTS.md or CLAUDE.md file in the root of your project.

3

Validate your setup

If you cannot create a rules file or can't invoke your agent, ask for help.

Checkpoint: Everyone has a text editor open and can use their agent.

SESSION BLOCK

What good acceptance criteria looks like

Bad vs Good Acceptance Criteria

Bad

Good

 Vague

 Specific

 Plausible-sounding but untestable

 Testable

 Over-specified on implementation detail

 No implementation detail

 Missing failure modes

 Covers at least one edge case

7 MINUTES

Individual — then share with the group

Apply the principle: can you recognize bad acceptance criteria?

Your Task

1. Find a story in your own backlog
2. Read the story and acceptance criteria
3. Compare the acceptance criteria versus good principles

The Critical Constraint

Are these things true of the acceptance criteria you see?

- They are specific
- They are testable
- There is no implementation detail
- They cover at least one edge case

SESSION BLOCK

RTCC

The structure every command in this program uses

R – Role

Who is the AI acting as? A specific, credentialed persona changes tone, vocabulary, and default assumptions.

T – Task

What specific action should it perform? Tasks should be verb-first and unambiguous.

C – Context

What background information does it need? What does it know about the situation, the codebase, the prior steps, the audience?

C – Constraints

What rules must the output follow? What should it never do? What format is required?

Why this order matters

Role and Task define the basic contract. Context shapes how the contract is fulfilled. Constraints are guardrails. Adding Constraints before establishing Role and Task often produces over-restricted outputs that satisfy the guardrails but miss the intent.

5 MINUTES

Practice applying the RTCC structure to a real problem, generating good acceptance criteria

Your Task

Write out what you would put for each part of the RTCC prompt formula for a command to create acceptance criteria.

This pre-work becomes the raw material for lab 2.

The Critical Constraint

Write out the instructions for each of the following:

- R: Who should be generating this Acceptance Criteria?
- T: What exactly are they doing?
- C: What does this persona need to know about the team's story format and domain?
- C: What must always be true of the output, and what should it never include?

SESSION BLOCK

EDD – Evaluation-Driven Development

Define success before you build

The EDD Discipline

Before building a command, define what good output looks like — specifically, in writing — so that you can evaluate whether the command is producing it.

Why This Matters

If you let the model define what is good, it will shape your expectations.

10 MINUTES

Individual — then share with the group

Avoid relying on the AI's perspective

Your Task

For an Acceptance Criteria command, write 4–6 specific evaluation criteria before building anything. Each criterion should be answerable as yes/no or countable.

These criteria become the evaluation standard used after Lab 3.

The Critical Constraint

Examples of a strong criterion: "Every Acceptance Criterion starts with Given/When/Then or a testable system behavior statement."

15 Minute Break

Lab 2: Build Your Acceptance Criteria Generation Command

45 MINUTES

Hands-on Lab

Build your acceptance criteria command

Use the pre-work that you have done so far as starter for your command.

Use the RTCC prework

R: Your Role element — the specific persona

T: Your Task element — what the command does, verb-first

C: Your Context element — the background the model needs

C: Your Constraints element — the rules the output must follow

Apply the Load-Bearing Rule as you build

For each instruction you write, ask: which criterion from Block 4 would fail if I removed this instruction? If no criterion would fail, don't add the instruction.

A command instruction that corresponds to no evaluation criterion is either testing something you haven't written down, or it's not doing anything at all.

Work apart, work together

You will spend 10 minutes working on it by yourself.

Then we will split you off into groups.

Checkpoint: At the end, you need to have a working Acceptance Criteria Generation command.

Lab 3: Use It – Acceptance Criteria Generation on Real Stories

30 MINUTES

Hands-on Lab

Apply this to your own work.

1

Run your acceptance criteria generation command on real stories in your actual backlog

Devs and testers, run it on stories that you'll be writing tests or test plan for.

2

Dev and Testing role participants

Run it on stories you'll be writing tests or test plans for — this is your upstream material

3

Generate, review, and evaluate each output against the criteria from Block 4

Use the Evaluation-Driven Development success criteria that you created before.

4

Record failures

Which criteria does the output pass? Which does it fail? What's the most common failure?

Checkpoint: You should have failures to discuss at the end of this lab.

15 Minute Break

SESSION BLOCK

Evaluate the Output and Apply the Load-Bearing Rule

Group evaluation and audit

10 MINUTES

Individual — then share with the group

Practice the act of evaluating the quality of acceptance criteria

Your Task

1. Share outputs across the room — anonymized if preferred
2. Group scores each output on the criteria from Block 4
3. What's the most common failure pattern in this output?
4. Key question for Dev and Testing roles: "Could you write a test from this Acceptance Criteria?" If not, why not? Name it.

The Critical Constraint

Use the 4-6 specific evaluation criteria you defined for acceptance criteria earlier. Share your criteria and how a story does not meet them.

10 MINUTES

Review your own command.

Your Task

Refactor your acceptance criteria command. For each instruction...

The Critical Constraint

1. Which of your evaluation criteria would fail if you removed this instruction?
2. If no criterion would fail, the instruction is dead weight — flag it for removal
3. Run the command again after removing flagged instructions and verify the criteria scores don't drop

SESSION BLOCK

Baseline Self-Assessment

Please complete the AI Adoption Maturity Assessment individually

The assessment covers 7 dimensions: Prompt Engineering & AI Interaction, Task Agents & Single-Box Automation, Workflow Orchestration & Coordination, Output Evaluation & Quality Judgment, Governance/Trust & Safety Nets, Trust & Working Identity, and Team & Organizational Practices

21 questions total; each answered A through D (1–4 points).

This is not graded and is not a tool to use against you. Please be honest with yourself and us.

SESSION BLOCK

Wrap-up

Block 7 | 10 Minutes | Discussion

- ✓ Work with your Embedded Engineer.
- ✓ Before week 2, use your Acceptance Criteria command on at least 3 real stories. Evaluate each output against the criteria you wrote today.
- ✓ Share the output, criteria scores, and failure modes with the Embedded Engineer.

Templates

SESSION BLOCK

[Block Name]

[Block Subtitle or Topic]

Block # | ## Minutes | [Format: Presentation / Activity / Lab / Discussion]

[Concept or Principle A]

[Explanation of what this means in practice. Keep it to 2–3 sentences. Concrete beats abstract.]

[Concept or Principle B]

[Explanation of what this means in practice. Keep it to 2–3 sentences. Concrete beats abstract.]

[Concept or Principle C]

[Explanation of what this means in practice. Keep it to 2–3 sentences. Concrete beats abstract.]

[Concept or Principle D]

[Explanation of what this means in practice. Keep it to 2–3 sentences. Concrete beats abstract.]

X

[Issue or Failure Mode A]

[Description of what this looks like in practice. Why does it happen?]

X

[Issue or Failure Mode B]

[Description of what this looks like in practice. Why does it happen?]

X

[Issue or Failure Mode C]

[Description of what this looks like in practice. Why does it happen?]

X

[Issue or Failure Mode D]

[Description of what this looks like in practice. Why does it happen?]

Discussion

[Discussion Prompt — Pose a question to the group.]

Share amongst yourselves 1–2 real examples relevant to this topic.

Concrete beats abstract.

MINUTES

Individual — then share with the group

[Brief framing sentence — what is this activity about and why does it matter?]

Your Task

1. [First step — what to do first]
2. [Second step — what to create or write]
3. [Third step — the key constraint or rule to apply]

The Critical Constraint

[Describe the key constraint that makes this activity non-trivial. This is the insight the trainer wants participants to discover through struggle.]

If participants do X, they are missing the point. Start over from Y.

MINUTES

GROUPS OF 3–4

Mixed roles in each group — no two from the same role

[Framing sentence: context for the group challenge, what they just experienced and why this matters.]

Your Group's Challenge

[State the core challenge: what must the group decide, produce, or determine?]

Ask yourselves:

- [Question 1 to guide discussion]
- [Question 2 to guide discussion]
- [Question 3 to guide discussion]

Use your lab results / prior work as evidence.

Group Deliverable

Each group shares back:

1. [Deliverable item 1]
2. [Deliverable item 2]
3. [Deliverable item 3 — often an open question]

[Note about how the outputs will be used by the Embedded Engineer or in subsequent sessions.]

Demo: [Approach A] vs. [Approach B]

[Approach A — the weaker/old way]

[Approach B — the structured/better way]

 [Absent or weak in Approach A]

 [Criterion 1 — e.g., Language/framework context]

 [Absent or weak in Approach A]

 [Criterion 2 — e.g., Specification provided]

 [Absent or weak in Approach A]

 [Criterion 3 — e.g., Failure mode instruction]

 [Absent or weak in Approach A]

 [Criterion 4 — e.g., Anti-pattern guard]

 [Absent or weak in Approach A]

 [Criterion 5 — e.g., Result quality]

 [Absent or weak in Approach A]

 [Criterion 6 — e.g., Traceability to requirements]

Lab #: [Lab Title]

MINUTES

Hands-on Lab

[Brief framing — what must be built or produced by the end of this lab?]

1

[Step 1 Title]

[What to do in this step. Be concrete — what exactly is the participant building, writing, or configuring?]

2

[Step 2 Title]

[Next step. Include any specific requirements or constraints. What must it contain or satisfy?]

3

[Step 3 Title]

[Third step. This might be a review, validation, or pairing checkpoint.]

4

[Step 4 Title]

[Optional fourth step. Could be an extension or a commit/save instruction.]

Checkpoint: [What to validate before moving on — a pair check, a commit, or a visible artifact.]

Lab #: [Lab Evaluation Title]

MINUTES

Hands-on Lab

[What to run or do in this lab. Specify the deliverable and the inputs to use.]

Evaluate each output against these criteria:

[Criterion 1 — maps to a named requirement or specification]

Y / N

[Criterion 2 — tests behavior, not implementation]

Y / N

[Criterion 3 — at least one failure/edge case covered]

Y / N

[Criterion 4 — would compile, run, or be executable as-is]

Y / N

Write it down: [Reflection prompt — e.g., what is the most common failure mode and your hypothesis about why?]

What does this session mean for your role specifically?

[Role 1]
[e.g., Product / Process]

[Question directed at this role — what does this session change or clarify for them?]

[What participants in this role should take away. Include the downstream consequence if relevant.]

[Role 2]
[e.g., Development]

[Question directed at this role — what do they need or what action do they take?]

[What participants in this role should take away. Include any concrete tool or artifact they are now responsible for.]

[Role 3]
[e.g., Testing]

[Question directed at this role — how does this change their work or coverage strategy?]

[What participants in this role should take away. Clarify what unit tests cover vs. what their test plan must cover.]

Session Artifacts — Every Participant Leaves With:



[Artifact 1 — a committed file, skill file, or configured tool]



[Artifact 2 — a written evaluation, notes, or output from a lab]



[Artifact 3 — something written: a hypothesis, a reflection, a decision]



[Artifact 4 — your group's deliverable: requirements, standards, or list]

This Week — Working with the Embedded Engineer

[Instructions for what to bring to the Embedded Engineer this week. What should participants do with the artifacts they built? What signal should they surface? How does this connect to Session #N?]